



Big-Step 증명나무 합성을 통한 정적 분석기 공격

프로그램이 아니라, 그 프로그램의 실행 의미를 합성한다

정원준 지도교수 이광근 서울대학교 프로그래밍 연구실 ROPAS



왜 증명나무를 합성하는가

프로그램을 먼저 합성하면, 그 프로그램에 실행 의미가 있는지조차 알 수 없다.

그래서 프로그램 대신 실행 의미 — Big-Step 증명나무 — 를 합성한다. 프로그램은 그 증명의 결론으로 따라 나온다.

프로그램이 멈추는지조차 미리 알 수 없다 (Rice's Theorem). 반대로 유한한 증명나무가 있다는 것은 곧 그 실행이 존재한다는 뜻이다.

01 문제 설정

분석기가 가짜 경로를 내거나 안전성을 잃는 가장 작은 프로그램을, 사람 없이 자동으로 찾는다.

분석기는 C를 분석하지만, 실제 분석은 C를 단순화한 CIL과 그로부터 만든 CFG 위에서 일어난다. 모든 C 문법이 공격에 필요한 것은 아니다.

실제 분석 경로



분석 대상은 C가 아니라, 그 아래로 내려간 CIL과 CFG다.

그래서 공격에 필요한 만큼만 담은 더 작은 언어 CIL--를 새로 정의한다.

찾아낸 공격은 분석기를 강화하는 지침이 되고, 거꾸로 난독화에도 쓸 수 있다.

02 언어: CIL--

대상 분석기는 Sparrow. C를 CIL 1.7.3으로 낮춘 뒤 그 위에서 분석한다. CIL--는 그 CIL의 부분집합이자 합성·실행·증명의 기준 언어다.

포함

int / unsigned int
직접 함수 호출 — 재귀와 상호재귀
if · loop · break · continue · return
포인터와 배열 문법

제외

cast · float · struct/union · 문자열
switch · goto · varargs · typedef · enum

cast-free

명시적 cast와 프론트엔드가 몰래 넣는 암묵 변환을 구분할 필요가 없도록, CastE를 아예 두지 않는다.

하나의 GADT, 두 개의 mode

Syntax.ground

hole 불가 — 실행 · 검증

Syntax.holed

ExpHole · StmtSeqHole

03 공격의 정의

main이 0을 반환하며 정상 종료한 실행만 비교한다. 그 시점에 살아 있는 모든 지역 memory binding이 관찰 대상.

안전성 실패

구체값 ≠ 분석 결과

정밀도 실패

분석 결과 ⊃ {구체값}

찾아낸 공격 예시

```
x = 1;
while (-x) { x = 0; }
x = x * x;
```

구체 실행 x = 0

분석 결과 x |-> [-inf, inf]

guard가 x가 아니라 -x라 종료 후 필터가 약하고, x * x는 두 피연산자가 같은 변수라는 상관관계를 잃는다.

* 이전 자체 분석기 엔진에서 발견

04 앞에서 뿌리로, 작은 조각부터 쌓아 올린다

```
int main() {
  int x;
  x = 1;
  return x;
}
```

EConst, LVar 같은 가장 작은 조각에서 시작해 뿌리까지 올라간다. 그림의 위에서 아래가 합성 순서다. 프로그램은 뿌리의 결론으로 따라 나온다.

[LVar] x => s0+0	[EConst] 1 => 1	[LVar] x => s0+0
[ISet] x = 1; => {x -> 1}	[ELval] x => 1	
[SInstr] instr[1] => Normal	[SReturnSome] return x; => Return(1)	
[BSeq] block[2] => Return(1)		
[FReturn] main() => Return(1)		
[PMainReturn] main() => 1		

proof_size(t) = 1 + sum of premise sizes

크기 순서대로 pool에 쌓는다

규칙마다 이미 만들어 둔 더 작은 조각만 꺼내 쓴다. 호출된 함수의 증명도 언제나 진부분나무이므로 재귀·상호재귀가 잘 정의된다.

메모리가 항상 따라 붙는다

완전한 프로그램은 스스로 메모리를 정하지만, 조각은 앞뒤 메모리의 관계만 안다. 메모리를 나중에 미루면 정지 문제에 빠지므로, 조각을 만들 때 전후 메모리를 확정한다.

05 실행 안 된 자리는 hole로

```
if (x || ?H1) {
  return 0;
} else {
  ?H2
}
```

?H1 short-circuit로 평가되지 않은 오른쪽 피연산자 — ExpHole

?H2 선택되지 않은 branch, 또는 return · break · continue 뒤에 잘린 block 꼬리 — StmtSeqHole

실행된 자리에는 hole이 남을 수 없다.

같은 정적 위치가 loop · 재귀로 여러 번 실행되면 그 조각들이 서로 맞아야 한다. 타입 추론에서와 같은 unification으로 맞추고, 나무와 substitution을 따로 들고 다닌다.

06 hole을 채워 분석기에 넣는다

hole 자리는 어차피 실행되지 않았다. 그래서 아무 코드나 채워 넣어도 이 증명나무가 말하는 실행 의미는 바뀌지 않는다. 문맥이 맞을 필요도 없다. 새로운 실행이 생기지 않도록 함수 호출만 넣지 않는다.



구체값 ≠ 분석 결과 → 안전성 공격

분석 결과 ⊃ {구체값} → 정밀도 공격

정밀도 공격은 튜링 완전한 부분집합이지만 하면 반드시 존재한다 (Rice's Theorem). 안전성 공격은 버그를 찾는 일이라, 언어 기능이 많을수록 유리하다.

07 정말 올바른 공격인가

공격이 성공했다고 말하려면, 분석기가 전제하는 실행 의미와 우리가 CIL--에 준 실행 의미가 같아야 한다.

아무도 실행 의미를 엄밀히 적어두지 않았다

C의 ISO 표준은 자연어로 되어 있고, CIL에도 엄밀한 실행 의미가 없다. 분석기 역시 sound in design이라고 말할 뿐 concrete semantics를 정의하지 않는다.

우리에게는 CIL--의 Big-Step 실행 의미가 있다. 비결정성이 남아 있으면 공격에 성공한 것처럼 보여도 사실은 비결정성 탓일 수 있으므로, 하나씩 없앤다.

예	C	CIL	CIL--
f() + g()	계산 순서가 정해지지 않음	호출이 식에 들어가지 않음	없음
i = i++ + 1;	undefined behavior	i++ 문법이 없음	없음
int x; use(x);	indeterminate value	indeterminate value	합성 시 만들지 않음
"a" == "a"	true / false 모두 가능	여전히 비결정적	문자열이 없음
char c = -1;	-1 또는 255	여전히 비결정적	char가 없음
malloc(n)	주소가 비결정적	fresh object	포인터가 없음